

Documentation for File Uploader Project

This documentation covers all relevant files found in your file upload project, explaining their purpose, structure, and usage. This project allows users to upload Excel files through a web interface, parses the data, and stores it in a SQLite database. The backend exposes a REST API that handles file uploads and data retrieval.

db.js

This module manages the SQLite database, including initialization and bulk-insertion logic. It ensures that the required directory and table exist and provides an interface for inserting uploaded spreadsheet rows into the database.

Main Features

- **Database Initialization**
 - Creates a `db` directory if not present.
 - Initializes a SQLite database named `data.db`.
- **Table Creation**
 - Ensures `upload_rows` table exists with columns: `id`, `col_a`, `col_b`, `col_c`.
- **Bulk Insert Function**
 - Efficiently inserts multiple rows from the uploaded Excel data.

Code Breakdown

```
import Database from 'better-sqlite3';
import { mkdirSync } from 'fs';
import { dirname, join } from 'path';
import { fileURLToPath } from 'url';
const __dirname = dirname(fileURLToPath(import.meta.url));
const dbDir = join(__dirname, 'db');
mkdirSync(dbDir, { recursive: true });

const db = new Database(join(dbDir, 'data.db'));

db.exec(CREATE TABLE IF NOT EXISTS upload_rows ( id INTEGER PRIMARY KEY AUTOINCREMENT, col_a TEXT, col_b TEXT, col_c TEXT ));

const insert = db.prepare('INSERT INTO upload_rows (col_a, col_b, col_c) VALUES (?, ?, ?)');
```

```
export function bulkInsert(rows) {
  const runMany = db.transaction((data) => {
    for (const r of data)
      insert.run(r.col_a, r.col_b, r.col_c);
  });
  runMany(rows);
}

export { db };
```

Database Schema

```
erDiagram
    upload_rows {
        INTEGER id PK "Primary key, auto-increment"
        TEXT col_a "First column"
        TEXT col_b "Second column"
        TEXT col_c "Third column"
    }
```

README.md

The `README.md` provides a concise instruction on how to run the project locally.

Content

```
# Upload Tech Test - Question 2
```

Run locally

```
1  npm install
2  npm start
3  </code></pre>
4  <pre><code>
5  - Purpose: Briefly explains how to start the server.
6  - Instructions:
7    1. Install dependencies: `npm install`
8    2. Start the server: `npm start`
9
10 ---
11
12 ## `package.json`
13
```

```
14 This file defines the Node.js project, its dependencies, scripts, and essential metadata.
15
16 ### Key Sections
17
18 | Field          | Description |
19 |-----|-----|
20 | name           | Project name (`file_uploader`) |
21 | version        | Project version (`1.0.0`) |
22 | description     | Short description (`File Upload`) |
23 | main           | Entry file (not actually used, as server.js is main) |
24 | type           | Module type (`module` for ES modules) |
25 | scripts        | Start, test, and dev commands |
26 | dependencies    | List of required dependencies |
27 | devDependencies | Nodemon for development server reloads |
28
29 ##### Dependencies
30
31 - **express** - Web framework for Node.js.
32 - **multer** - Middleware for handling file uploads.
33 - **better-sqlite3** - SQLite binding for fast database operations.
34 - **xlsx** - Parses and writes Excel files.
35 - **dotenv** - Loads environment variables from `.env`.
36
37 ##### DevDependencies
38
39 - **nodemon** - Restarts the server on code changes.
40
41 ### Install All Dependencies
42
43 ```packagemanagers
44 {
45   &quot;commands&quot;: {
46     &quot;npm&quot;: &quot;npm install&quot;,
47     &quot;yarn&quot;: &quot;yarn install&quot;,
48     &quot;pnpm&quot;: &quot;pnpm install&quot;,
49     &quot;bun&quot;: &quot;bun install&quot;
50   }
51 }
52 </code></pre>
53 <hr />
54 <h2><code>package-lock.json</code></h2>
55 <p>This file ensures deterministic dependency versions for builds. It records the exact dependency tree
56 <ul>
57 <li><strong>Purpose:</strong> Guarantees all developers and deployments use the same dependency version
58 <li><strong>Usage:</strong> Managed automatically; do not edit manually.</li>
59 </ul>
60 <hr />
61 <h2><code>server.js</code></h2>
62 <p>This is the main backend server file. It defines the Express application, sets up middleware, config
63 <h3>Key Functions</h3>
64 <ul>
65 <li>Serves static files from the <code>public</code> directory.</li>
66 <li><strong>Health check route</strong> at <code>/</code>.</li>
67 <li><strong>Excel file upload API</strong> at <code>/api/upload</code>.</li>
68 <li><strong>Data retrieval API</strong> at <code>/api/rows</code>.</li>
69 <li>Starts the server on the specified port (default: <code>4002</code>).</li>
70 </ul>
71 <h3>Highlights</h3>
72 <ul>
73 <li>Uses <code>multer</code> for file handling, storing uploads in <code>uploads/</code>.</li>
```

```

74 <li>Reads <code>.xlsx</code> files with the <code>xlsx</code> package.</li>
75 <li>Maps columns <code>A</code>, <code>B</code>, <code>C</code> (or <code>col_a</code>, <code>col_b</code>)</li>
76 <li>Inserts rows into SQLite using <code>bulkInsert</code>.</li>
77 <li>Returns the number of inserted rows as the response.</li>
78 <li>Handles errors and returns appropriate status codes.</li>
79 </ul>
80 <h3>API Endpoint Documentation</h3>
81 <h4>Health Check</h4>
82 <pre><code class="language-api">{
83     &quot;title&quot;;: &quot;Health Check&quot;;,
84     &quot;description&quot;;: &quot;Returns a simple status message to verify the API is running.&quot;;,
85     &quot;method&quot;;: &quot;GET&quot;;,
86     &quot;baseUrl&quot;;: &quot;http://localhost:4002&quot;;,
87     &quot;endpoint&quot;;: &quot;/&quot;;,
88     &quot;headers&quot;;: [],
89     &quot;queryParams&quot;;: [],
90     &quot;pathParams&quot;;: [],
91     &quot;bodyType&quot;;: &quot;none&quot;;,
92     &quot;requestBody&quot;;: &quot;&quot;;,
93     &quot;responses&quot;;: {
94         &quot;200&quot;;: {
95             &quot;description&quot;;: &quot;API is operational&quot;;,
96             &quot;body&quot;;: &quot;UPLOAD-TEST API OK&quot;;
97         }
98     }
99 }
100 </code></pre>
101 <h4>Upload Excel File</h4>
102 <pre><code class="language-api">{
103     &quot;title&quot;;: &quot;Upload Excel File&quot;;,
104     &quot;description&quot;;: &quot;Uploads an Excel (.xlsx) file and imports its rows into the database&quot;;,
105     &quot;method&quot;;: &quot;POST&quot;;,
106     &quot;baseUrl&quot;;: &quot;http://localhost:4002&quot;;,
107     &quot;endpoint&quot;;: &quot;/api/upload&quot;;,
108     &quot;headers&quot;;: [
109         {
110             &quot;key&quot;;: &quot;Content-Type&quot;;,
111             &quot;value&quot;;: &quot;multipart/form-data&quot;;,
112             &quot;required&quot;;: true
113         }
114     ],
115     &quot;queryParams&quot;;: [],
116     &quot;pathParams&quot;;: [],
117     &quot;bodyType&quot;;: &quot;form&quot;;,
118     &quot;formData&quot;;: [
119         {
120             &quot;key&quot;;: &quot;file&quot;;,
121             &quot;value&quot;;: &quot;The .xlsx file to upload&quot;;,
122             &quot;required&quot;;: true
123         }
124     ],
125     &quot;responses&quot;;: {
126         &quot;200&quot;;: {
127             &quot;description&quot;;: &quot;Success - Number of rows inserted&quot;;,
128             &quot;body&quot;;: &quot;{\n  \&quot;inserted\&quot;;: 42\n}&quot;;
129         },
130         &quot;500&quot;;: {
131             &quot;description&quot;;: &quot;Server error (e.g. invalid Excel file)&quot;;,
132             &quot;body&quot;;: &quot;{\n  \&quot;error\&quot;;: \&quot;Error message\&quot;;\n}&quot;;
133         }
134     }
135 }
136 </code></pre>

```

```

134     }
135 }
136 </code></pre>
137 <h4>List Inserted Rows</h4>
138 <pre><code class="language-api">{
139     &quot;title&quot;;: &quot;List Inserted Rows&quot;;,
140     &quot;description&quot;;: &quot;Retrieves all rows that were uploaded and stored in the database.&quot;;,
141     &quot;method&quot;;: &quot;GET&quot;;,
142     &quot;baseUrl&quot;;: &quot;http://localhost:4002&quot;;,
143     &quot;endpoint&quot;;: &quot;/api/rows&quot;;,
144     &quot;headers&quot;;: [],
145     &quot;queryParams&quot;;: [],
146     &quot;pathParams&quot;;: [],
147     &quot;bodyType&quot;;: &quot;none&quot;;,
148     &quot;requestBody&quot;;: &quot;&quot;;,
149     &quot;responses&quot;;: {
150         &quot;200&quot;;: {
151             &quot;description&quot;;: &quot;Returns an array of all uploaded rows&quot;;,
152             &quot;body&quot;;: &quot;[\n  {\n    \&quot;id\&quot;;: 1,\n    \&quot;col_a\&quot;;: \&quot;;,
153         }
154     }
155 }
156 </code></pre>
157 <h3>API Flow</h3>
158 <pre><code class="language-mermaid">sequenceDiagram
159     participant User
160     participant Browser
161     participant ExpressServer
162     participant Database
163
164     User->&gt;Browser: Selects and submits Excel file
165     Browser->&gt;ExpressServer: POST /api/upload (multipart/form-data)
166     ExpressServer->&gt;ExpressServer: Parse XLSX, map columns
167     ExpressServer->&gt;Database: bulkInsert(rows)
168     Database--&gt;&gt;ExpressServer: Confirmation
169     ExpressServer--&gt;&gt;Browser: JSON { inserted: n }
170     Browser--&gt;&gt;User: Shows success/failure message
171 </code></pre>
172 <hr />
173 <h2><code>upload.html</code></h2>
174 <p>This file implements the user interface for uploading Excel files. It uses a modern, card-based des
175 <h3>Features</h3>
176 <ul>
177 <li><strong>File Input:</strong> Accepts <code>.xlsx</code> files only.</li>
178 <li><strong>Custom File Picker UI:</strong> Replaces default file input.</li>
179 <li><strong>AJAX Upload:</strong> Submits file via JavaScript <code>fetch</code> API.</li>
180 <li><strong>Feedback:</strong> Displays upload progress, success, or error messages.</li>
181 <li><strong>Styling:</strong> Responsive, dark-themed, with accent colors.</li>
182 </ul>
183 <h3>Main UI Structure</h3>
184 <table>
185 <thead>
186 <tr>
187 <th>Element</th>
188 <th>Purpose</th>
189 </tr>
190 </thead>
191 <tbody>
192 <tr>
193 <td><code>.card</code></td>

```

```

194 <td>Main container for the form and status</td>
195 </tr>
196 <tr>
197 <td><code>form#upForm</code></td>
198 <td>The file upload form</td>
199 </tr>
200 <tr>
201 <td><code>input#fileInp</code></td>
202 <td>Hidden file input, accepts <code>.xlsx</code></td>
203 </tr>
204 <tr>
205 <td><code>.file-label</code></td>
206 <td>Clickable label for file input</td>
207 </tr>
208 <tr>
209 <td><code>.btn</code></td>
210 <td>Submit button</td>
211 </tr>
212 <tr>
213 <td><code>#log</code></td>
214 <td>Displays success/error messages</td>
215 </tr>
216 </tbody>
217 </table>
218 <h3>Key JavaScript</h3>
219 <ul>
220 <li>Changes label text to the selected file name.</li>
221 <li>Handles form submission via AJAX.</li>
222 <li>Displays number of rows inserted or error.</li>
223 </ul>
224 <h3>Example UI Flow</h3>
225 <pre><code class="language-mermaid">flowchart TD
226     A[User chooses Excel file] --> B{File selected?}
227     B -- No --> C[Prompt: Choose file...]
228     B -- Yes --> D[Show file name on label]
229     D --> E[User clicks Import Data]
230     E --> F[AJAX POST to /api/upload]
231     F --> G{Success?}
232     G -- Yes --> H[Show "Inserted X rows"]
233     G -- No --> I[Show error message]
234 </code></pre>
235 <hr />
236 <h2><code>index.html</code></h2>
237 <p>This is the landing page for the project. It displays a simple card with a button to access the upl
238 <h3>Features</h3>
239 <ul>
240 <li><strong>Card Layout:</strong> Clean, minimal UI.</li>
241 <li><strong>Launch Button:</strong> Directs user to <code>/upload.html</code>.</li>
242 <li><strong>Status Message:</strong> Confirms backend is running.</li>
243 </ul>
244 <h3>Main Elements</h3>
245 <table>
246 <thead>
247 <tr>
248 <th>Element</th>
249 <th>Purpose</th>
250 </tr>
251 </thead>
252 <tbody>
253 <tr>

```

```
254 <td><code>.card</code></td>
255 <td>Container for all content</td>
256 </tr>
257 <tr>
258 <td><code>h1</code></td>
259 <td>Title</td>
260 </tr>
261 <tr>
262 <td><code>p</code></td>
263 <td>Sub-title / status message</td>
264 </tr>
265 <tr>
266 <td><code>a.btn</code></td>
267 <td>Button to open upload form</td>
268 </tr>
269 </tbody>
270 </table>
271 <hr />
272 <h2>System Overview</h2>
273 <h3>File/Module Relationships</h3>
274 <pre><code class="language-mermaid">flowchart TD
275     subgraph Client
276         upload_html[upload.html]
277         index_html[index.html]
278     end
279     subgraph Server
280         server_js[server.js]
281         db_js[db.js]
282         data_db[data.db]
283     end
284     subgraph Config
285         package_json[package.json]
286         package_lock_json[package-lock.json]
287         README_md[README.md]
288     end
289
290     upload_html -- AJAX /api/upload --> server_js
291     server_js -- import --> db_js
292     db_js -- reads/writes --> data_db
293     index_html -- /upload.html link --> upload_html
294 </code></pre>
295 <hr />
296 <h2>Summary Table</h2>
297 <table>
298 <thead>
299 <tr>
300 <th>File</th>
301 <th>Purpose</th>
302 </tr>
303 </thead>
304 <tbody>
305 <tr>
306 <td><code>db.js</code></td>
307 <td>Database setup and bulk insert logic</td>
308 </tr>
309 <tr>
310 <td><code>README.md</code></td>
311 <td>Quickstart instructions</td>
312 </tr>
313 <tr>
```

```
314 <td><code>package.json</code></td>
315 <td>Project metadata, dependencies, scripts</td>
316 </tr>
317 <tr>
318 <td><code>package-lock.json</code></td>
319 <td>Exact dependency versions for reproducible installs</td>
320 </tr>
321 <tr>
322 <td><code>server.js</code></td>
323 <td>Main backend file, defines API and server logic</td>
324 </tr>
325 <tr>
326 <td><code>upload.html</code></td>
327 <td>User-facing form for uploading Excel files</td>
328 </tr>
329 <tr>
330 <td><code>index.html</code></td>
331 <td>Landing/status page for the project</td>
332 </tr>
333 </tbody>
334 </table>
335 <hr />
336 <h2>□ Getting Started</h2>
337 <p>To run the project locally:</p>
338 <ol>
339 <li><strong>Install dependencies:</strong>
340 <pre><code class="language-bash">npm install
341 </code></pre>
342 </li>
343 <li><strong>Start the server:</strong>
344 <pre><code class="language-bash">npm start
345 </code></pre>
346 </li>
347 <li><strong>Visit:</strong> <a href="http://localhost:4002">http://localhost:4002</a> in your browser.
348 </li>
349 </ol>
350 <hr />
351 <h2>□ API Summary</h2>
352 <ul>
353 <li><strong>GET /</strong> – Health check</li>
354 <li><strong>POST /api/upload</strong> – Upload Excel file (multipart/form-data)</li>
355 <li><strong>GET /api/rows</strong> – Retrieve inserted rows as JSON</li>
356 </ul>
357 <hr />
358 <h2>□ Notes</h2>
359 <ul>
360 <li>Only <code>.xlsx</code> files are accepted.</li>
361 <li>Database schema is fixed: only columns A/B/C or col_a/col_b/col_c are imported.</li>
362 <li>The UI provides clear feedback for each upload attempt.</li>
363 <li>The backend is easily extensible for more columns or advanced validation.</li>
364 </ul>
365 <hr />
366 <p><strong>This documentation should help developers, testers, and users understand, extend, and opera
```